

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 3**



Single Linked List

Oleh:

Muhammad Hamdani

2510817310019

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 3

Laporan Praktikum Algoritma & Struktur Data Modul 3: Single Linked List ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Hamdani
NIM : 2510817310019

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Fadhil Syahdama Mahatma Putra
NIM. 2410817210026

Andreyan Rizky Baskara, S.Kom., M.Kom
NIP. 199307032019031011

DAFTAR ISI

LEMBAR PENGESAHAN	i
DAFTAR ISI.....	ii
DAFTAR GAMBAR.....	iii
DAFTAR TABEL.....	iv
SOAL 1	1
Pembahasan.....	11
SOAL 2	15
Pembahasan.....	15
SOAL 3	16
Pembahasan.....	16
SOAL 4	17
Pembahasan.....	17
SOAL 5	18
Pembahasan.....	18
SOAL 6	19
Pembahasan.....	19
SOAL 7	21
Pembahasan.....	21
SOAL 8	22
Pembahasan.....	22
SOAL 9	24
Pembahasan.....	24
SOAL 10	25
Pembahasan.....	25
TAUTAN GITHUB	26

DAFTAR GAMBAR

Gambar 1.Screenshot Output Program	11
Gambar 2.Tampilan Nilain Yang Diinput Dari Depan	15
Gambar 3.Tampilan Yang Diinput Dari Belakang	16
Gambar 4.Pencarian Nilai 2 pada Linked List.....	17
Gambar 5.Pencarian Nilai 7 pada Linked List.....	18
Gambar 6.Tampilan Linked List Setelah Di Hapus	19
Gambar 7.Tampilan Linked List Setelah Setiap Nilai 3 Dihapus	21
Gambar 8.Tampilan Isi dari Linked List.....	22

DAFTAR TABEL

TABEL 1. Source Code Linked List.....	1
---------------------------------------	---

SOAL 1

Cobalah program berikut, running, simpan program, dan screenshoot hasil running !

TABEL 1. Source Code Linked List

1.	#include <iostream>
2.	#include <conio.h>
3.	#include <stdlib.h>
4.	
5.	using namespace std;
6.	
7.	typedef struct TNode {
8.	string data;
9.	TNode *next;
10.	} TNode;
11.	
12.	TNode *head, *tail;
13.	
14.	int pil;
15.	string dataBaru, dataHapus; // Menggunakan dataHapus sesuai instruksi perbaikan
16.	
17.	// Deklarasi Fungsi
18.	void init();
19.	int isEmpty();
20.	void tambahDepan();
21.	void tambahBelakang();
22.	void hapusDepan();
23.	void hapusBelakang();
24.	void tampilkan();
25.	void reset();
26.	void cariData();
27.	void hapusData();
28.	void sisipkanSebelum();
29.	void sisipkanSetelah();
30.	
31.	int main()
32.	{
33.	init();
34.	
35.	do {
36.	cout<<"Single Linked List Circular (SLLC) "<<endl;

```

37. cout<<"===== "<<endl;
38.     cout<<"1. Tambah Depan"<<endl;
39.     cout<<"2. Tambah Belakang"<<endl;
40.     cout<<"3. Hapus Depan"<<endl;
41.     cout<<"4. Hapus Belakang"<<endl;
42.     cout<<"5. Tampilkan Data"<<endl;
43.     cout<<"6. Hapus Semua Elemen"<<endl;
44.     cout<<"7. Cari Data"<<endl;
45.     cout<<"8. Hapus Setiap Data Tertentu"<<endl;
46.     cout<<"9. Sisipkan Node/Data Baru Sebelum
Data Tertentu"<<endl;
47.     cout<<"10. Sisipkan Node/Data Baru Setelah
Data Tertentu"<<endl;
48.     cout<<"11. Quit"<<endl;
49.     cout<<"Pilihan : ";
50.
51.     if (!(cin >> pil)) {
52.         cin.clear();
53.         cin.ignore(1000, '\n');
54.         continue;
55.     }
56.
57.     switch(pil) {
58.         case 1:
59.             tambahDepan();
60.             break;
61.         case 2:
62.             tambahBelakang();
63.             break;
64.         case 3:
65.             hapusDepan();
66.             if(!dataHapus.empty())
67.                 cout<<"Data \ "<<dataHapus<<"\ "
yang berada di depan telah berhasil dihapus."<<endl;
68.             break;
69.         case 4:
70.             hapusBelakang();
71.             if(!dataHapus.empty())
72.                 cout<<"Data \ "<<dataHapus<<"\ "
yang berada di belakang telah berhasil
dihapus."<<endl;
73.             break;

```

```

74.         case 5:
75.             tampilkan();
76.             break;
77.         case 6:
78.             reset();
79.             break;
80.         case 7:
81.             cariData();
82.             break;
83.         case 8:
84.             hapusData();
85.             break;
86.         case 9:
87.             sisipkanSebelum();
88.             break;
89.         case 10:
90.             sisipkanSetelah();
91.             break;
92.         case 11:
93.             cout<<"\nTERIMA KASIH"<<endl;
94.             cout<<"Program was made Muhammad
Hamdani (2510817310019)."<<endl;
95.             break;
96.         default:
97.             cout<<"\nPilihan                tidak
tersedia!"<<endl;
98.         }
99.         if(pil != 11) {
100.             cout<<"\nPress any key to continue!";
101.             getch();
102.         }
103.
104.     } while (pil != 11);
105.
106.     return 0;
107. }
108.
109. void init(){
110.     head = NULL;
111.     tail = NULL;
112. }
113.
114. int isEmpty() {

```

```

115.     if(head == NULL) return 1;
116.     else return 0;
117. }
118.
119. void tambahDepan() {
120.     cout<<"Masukkan data : ";
121.     TNode *baru = new TNode;
122.     cin>>dataBaru;
123.     baru->data = dataBaru;
124.     baru->next = baru;
125.
126.     if(isEmpty() == 1) {
127.         head = baru;
128.         tail = baru;
129.         tail->next = head;
130.     } else {
131.         baru->next = head;
132.         head = baru;
133.         tail->next = head;
134.     }
135.     cout << "Data \\"<<dataBaru<<"\" berhasil
dimasukkan di bagian depan."<<endl;
136. }
137.
138. void tambahBelakang() {
139.     cout<<"Masukkan data : ";
140.     TNode *baru = new TNode;
141.     cin>>dataBaru;
142.     baru->data = dataBaru;
143.     baru->next = baru;
144.
145.     if(isEmpty() == 1) {
146.         head = baru;
147.         tail = baru;
148.         tail->next = head;
149.     } else {
150.         tail->next = baru;
151.         tail = baru;
152.         tail->next = head;
153.     }
154.     cout << "Data \\"<<dataBaru<<"\" berhasil
dimasukkan di bagian belakang."<<endl;
155. }

```

```

156.
157. void hapusDepan() {
158.     dataHapus = ""; // Reset variabel
159.     if(isEmpty() == 0) {
160.         TNode *hapus = head;
161.         dataHapus = hapus->data;
162.
163.         if(head != tail) {
164.             head = head->next;
165.             tail->next = head;
166.         } else {
167.             init();
168.         }
169.         delete hapus;
170.     } else {
171.         cout<<"Tidak terdapat data pada Linked
List."<<endl;
172.     }
173. }
174.
175. void hapusBelakang() {
176.     dataHapus = ""; // Reset variabel
177.     if(isEmpty() == 0) {
178.         TNode *hapus = tail;
179.         TNode *newTail;
180.         dataHapus = hapus->data;
181.
182.         if(head != tail) {
183.             newTail = head;
184.             while(newTail->next != tail) {
185.                 newTail = newTail->next;
186.             }
187.             tail = newTail;
188.             tail->next = head;
189.         } else {
190.             init();
191.         }
192.         delete hapus;
193.     } else {
194.         cout<<"Tidak terdapat data pada Linked
List."<<endl;
195.     }
196. }

```

```

197.
198. void tampilkan() {
199.     if(isEmpty() == 0) {
200.         TNode *bantu = head;
201.         cout << "Data: ";
202.         do {
203.             cout<<bantu->data<<' ';
204.             bantu = bantu->next;
205.         } while(bantu != head);
206.         cout<<endl;
207.     } else {
208.         cout<<"Tidak terdapat data pada Linked
List."<<endl;
209.     }
210. }
211.
212. void reset() {
213.     if(isEmpty() == 0) {
214.         TNode *bantu = head;
215.         TNode *hapusNode;
216.
217.         do {
218.             hapusNode = bantu;
219.             bantu = bantu->next;
220.             delete hapusNode;
221.         } while(bantu != head);
222.
223.         init();
224.         cout<<"Seluruh elemen pada Linked List telah
dibersihkan."<<endl;
225.     } else {
226.         cout<<"Tidak terdapat data pada Linked
List."<<endl;
227.     }
228. }
229.
230. void cariData() {
231.     if(isEmpty() == 0) {
232.         string cari;
233.         cout<<"Masukkan data yang ingin dicari : ";
234.         cin>>cari;
235.
236.         TNode *bantu = head;

```

```

237.         TNode *bantuTampilkan;
238.         bool apaDitemukan = false;
239.
240.         do {
241.             if(cari == bantu->data){
242.                 cout<<"Setiap data yang berada di
dalam tanda kurung siku ([...]) merupakan data yang
anda cari."<<endl;
243.                 cout<<"Linked List : ";
244.                 bantuTampilkan = head;
245.
246.                 do {
247.                     if(cari == bantuTampilkan-
>data)
248.                         cout<<"["<<bantuTampilkan-
>data<<"] ";
249.                     else
250.                         cout<<bantuTampilkan-
>data<<' ';
251.                     bantuTampilkan =
bantuTampilkan->next;
252.                 } while(bantuTampilkan != head);
253.
254.                 apaDitemukan = true;
255.                 cout<<endl;
256.                 break;
257.             }
258.             bantu = bantu->next;
259.         } while(bantu != head);
260.
261.         if(!apaDitemukan)
262.             cout<<"Data \<<cari<<"\<<" tidak
ditemukan pada Linked List."<<endl;
263.         } else {
264.             cout<<"Tidak terdapat data pada Linked
List."<<endl;
265.         }
266.     }
267.
268. void hapusData() {
269.     if(isEmpty() == 0) {
270.         string cari;
271.         cout<<"Masukkan data yang ingin dihapus : ";

```

```

272.         cin>>cari;
273.
274.         TNode *bantu = head;
275.         TNode *sebelum = tail;
276.         TNode *hapusArr[255];
277.         int hitung = 0;
278.         bool apaDitemukan = false;
279.
280.         do {
281.             TNode *bantu2 = bantu;
282.             if(cari == bantu->data){
283.                 hapusArr[hitung++] = bantu;
284.                 apaDitemukan = true;
285.                 if(bantu != head && bantu != tail) {
286.                     sebelum->next = bantu->next;
287.                     bantu2 = sebelum;
288.                 }
289.             }
290.             sebelum = bantu2;
291.             bantu = bantu->next;
292.         } while(bantu != head);
293.
294.         if(apaDitemukan) {
295.             for(int i = 0; i < hitung; i++) {
296.                 if(hapusArr[i] == head){
297.                     hapusDepan();
298.                 } else if(hapusArr[i] == tail) {
299.                     hapusBelakang();
300.                 } else {
301.                     delete hapusArr[i];
302.                 }
303.             }
304.             cout<<"Setiap data \""<<cari<<"\" yang
terdapat pada Linked List telah dihapus."<<endl;
305.         } else {
306.             cout<<"Data        \""<<cari<<"\"        tidak
ditemukan pada Linked List."<<endl;
307.         }
308.         } else {
309.             cout<<"Tidak terdapat data pada Linked
List."<<endl;
310.         }
311.     }

```

```

312.
313. void sisipkanSebelum() {
314.     if(isEmpty() == 0) {
315.         TNode *bantu = head;
316.         TNode *sebelum = tail;
317.         string nextData;
318.         bool apaAda = false;
319.
320.         cout<<"Sisipkan data baru sebelum data : ";
321.         cin>>nextData;
322.
323.         do {
324.             if(nextData == bantu->data){
325.                 apaAda = true;
326.                 break;
327.             } else {
328.                 sebelum = bantu;
329.                 bantu = bantu->next;
330.             }
331.         } while(bantu != head);
332.
333.         if(apaAda) {
334.             cout<<"Masukkan data yang ingin
ditambahkan : ";
335.             cin>>dataBaru;
336.
337.             TNode *baru = new TNode;
338.             baru->data = dataBaru;
339.             baru->next = bantu;
340.             sebelum->next = baru;
341.
342.             if(bantu == head) head = baru;
343.
344.             cout << "Data \""<<dataBaru<<"\"
berhasil disisipkan sebelum data
\""<<nextData<<"\"."<<endl;
345.         } else {
346.             cout<<"Tidak terdapat data
\""<<nextData<<"\" pada Linked List."<<endl;
347.         }
348.     } else {
349.         cout<<"Tidak terdapat data pada Linked
List."<<endl;

```

```

350.     }
351. }
352.
353. void sisipkanSetelah() {
354.     if(isEmpty() == 0) {
355.         TNode *bantu = head;
356.         string prevData;
357.         bool apaAda = false;
358.
359.         cout<<"Sisipkan data baru setelah data : ";
360.         cin>>prevData;
361.
362.         do {
363.             if(prevData == bantu->data){
364.                 apaAda = true;
365.                 break;
366.             } else {
367.                 bantu = bantu->next;
368.             }
369.         } while(bantu != head);
370.
371.         if(apaAda) {
372.             cout<<"Masukkan data yang ingin
ditambahkan : ";
373.             cin>>dataBaru;
374.
375.             TNode *baru = new TNode;
376.             baru->data = dataBaru;
377.             baru->next = bantu->next;
378.             bantu->next = baru;
379.
380.             if(bantu == tail) tail = baru;
381.
382.             cout << "Data \""<<dataBaru<<"\"
berhasil disisipkan setelah data
\"<<prevData<<"\"."<<endl;
383.         } else {
384.             cout<<"Tidak terdapat data
\"<<prevData<<"\" pada Linked List."<<endl;
385.         }
386.     } else {
387.         cout<<"Tidak terdapat data pada Linked
List."<<endl;

```

388.	}
389.	}

Output Program

```

PROBLEMS OUTPUT TERMINAL ...
cppdbg: MuhammadHamdani2510817310019_modul3.exe + v [ ] [ ] ...

9. Sisipkan Node/Data Baru Sebelum Data Tertentu
10. Sisipkan Node/Data Baru Setelah Data Tertentu
11. Quit
Pilihan : 1
Masukkan data : 4
Data "4" berhasil dimasukkan di bagian depan.

Press any key to continue!Single Linked List Circular (SLLC)
=====
1. Tambah Depan
2. Tambah Belakang
3. Hapus Depan
4. Hapus Belakang
5. Tampilkan Data
6. Hapus Semua Elemen
7. Cari Data
8. Hapus Setiap Data Tertentu
9. Sisipkan Node/Data Baru Sebelum Data Tertentu
10. Sisipkan Node/Data Baru Setelah Data Tertentu
11. Quit
Pilihan : 2
Masukkan data : 5
Data "5" berhasil dimasukkan di bagian belakang.

Press any key to continue!Single Linked List Circular (SLLC)
=====
1. Tambah Depan
2. Tambah Belakang
3. Hapus Depan
4. Hapus Belakang
5. Tampilkan Data
6. Hapus Semua Elemen
7. Cari Data
8. Hapus Setiap Data Tertentu
9. Sisipkan Node/Data Baru Sebelum Data Tertentu
10. Sisipkan Node/Data Baru Setelah Data Tertentu
11. Quit
Pilihan : 5
Data: 4 5

```

Gambar 1. Screenshot Output Program

Pembahasan

Pada baris [1] sampai [4] terdapat `#include` yang digunakan untuk memanggil library yang dibutuhkan dalam program. Library `<iostream>` digunakan untuk proses input dan output seperti `cin` dan `cout`. Library `<string>` digunakan agar program dapat menggunakan tipe data string. Library `<conio.h>` digunakan untuk fungsi

getch() yang memungkinkan program membaca input keyboard tanpa harus menekan tombol Enter. Sedangkan <stdlib.h> digunakan untuk fungsi-fungsi umum seperti manajemen memori, konversi data, dan kontrol program.

Pada baris [6] terdapat perintah using namespace std; yang berfungsi untuk mempermudah penulisan kode sehingga tidak perlu menuliskan std:: sebelum fungsi standar seperti cout dan cin.

Pada baris [8] sampai [11] terdapat deklarasi struct TNode. Struct ini digunakan sebagai representasi node dalam linked list. Di dalam struct terdapat variabel data bertipe string yang berfungsi untuk menyimpan nilai atau isi dari node. Selain itu terdapat pointer next yang berfungsi untuk menunjuk ke node berikutnya. Karena menggunakan konsep circular linked list, maka node terakhir akan menunjuk kembali ke node pertama (head).

Pada baris [13] terdapat deklarasi pointer head dan tail. Pointer head digunakan untuk menunjuk node pertama dalam linked list, sedangkan pointer tail digunakan untuk menunjuk node terakhir. Kedua pointer ini sangat penting dalam proses manipulasi data seperti penambahan dan penghapusan node.

Pada baris [15] sampai [16] terdapat variabel global yaitu pil yang digunakan untuk menyimpan pilihan menu dari pengguna, serta dataBaru dan dataHapus yang digunakan untuk menyimpan data yang akan ditambahkan dan data yang dihapus.

Pada baris [18] sampai [28] terdapat deklarasi fungsi (prototype). Fungsi-fungsi ini meliputi berbagai operasi pada linked list seperti penambahan data, penghapusan data, pencarian data, serta penyisipan node.

Pada baris [30] sampai [99] terdapat fungsi main() yang merupakan bagian utama dari program. Pada bagian ini program akan menampilkan menu pilihan kepada pengguna. Menu terdiri dari beberapa opsi seperti tambah data, hapus data, tampilkan data, reset, cari data, dan sisip data. Program menggunakan perulangan do-while sehingga menu akan terus ditampilkan sampai pengguna memilih opsi keluar (pilihan 11).

Input dari pengguna akan diproses menggunakan struktur switch-case, di mana setiap pilihan akan memanggil fungsi tertentu sesuai dengan operasi yang dipilih. Selain itu terdapat getch() yang digunakan untuk menahan layar sebelum kembali ke menu.

Pada baris [101] sampai [104] terdapat fungsi init() yang digunakan untuk menginisialisasi linked list dalam kondisi kosong. Fungsi ini akan mengatur pointer head dan tail menjadi NULL.

Pada baris [106] sampai [109] terdapat fungsi isEmpty() yang digunakan untuk mengecek apakah linked list dalam keadaan kosong atau tidak. Jika head bernilai NULL, maka fungsi akan mengembalikan nilai 1 (kosong), jika tidak maka mengembalikan nilai 0 (tidak kosong).

Pada baris [111] sampai [128] terdapat fungsi tambahDepan() yang digunakan untuk menambahkan node baru di bagian depan linked list. Jika linked list dalam keadaan kosong, maka node baru akan menjadi head sekaligus tail. Jika tidak kosong, maka node baru akan ditempatkan sebelum head dan kemudian dijadikan sebagai head yang baru. Pointer tail->next juga akan diperbarui agar tetap menunjuk ke head.

Pada baris [130] sampai [147] terdapat fungsi tambahBelakang() yang digunakan untuk menambahkan node di bagian belakang linked list. Jika linked list kosong, maka node baru akan menjadi head dan tail. Jika tidak, maka node baru akan ditambahkan setelah tail, kemudian tail akan diperbarui menjadi node baru, dan tail->next akan diarahkan kembali ke head.

Pada baris [149] sampai [165] terdapat fungsi hapusDepan() yang digunakan untuk menghapus node pertama. Jika hanya terdapat satu node, maka linked list akan dikosongkan dengan memanggil fungsi init(). Jika lebih dari satu node, maka head akan dipindahkan ke node berikutnya, dan tail->next akan diperbarui agar tetap menunjuk ke head.

Pada baris [167] sampai [189] terdapat fungsi hapusBelakang() yang digunakan untuk menghapus node terakhir. Proses dilakukan dengan mencari node sebelum tail,

kemudian menjadikannya sebagai tail yang baru. Setelah itu, pointer tail->next akan diarahkan kembali ke head.

Pada baris [191] sampai [203] terdapat fungsi tampilkan() yang digunakan untuk menampilkan seluruh isi linked list. Proses penelusuran dilakukan menggunakan perulangan do-while karena sifat linked list yang circular, sehingga traversal dimulai dari head dan berhenti ketika kembali ke head.

Pada baris [205] sampai [222] terdapat fungsi reset() yang digunakan untuk menghapus seluruh node dalam linked list. Node akan dihapus satu per satu menggunakan perulangan hingga semua node terhapus, kemudian dilakukan inisialisasi ulang dengan fungsi init().

Pada baris [224] sampai [264] terdapat fungsi cariData() yang digunakan untuk mencari data tertentu dalam linked list. Jika data ditemukan, maka data tersebut akan ditampilkan dengan tanda kurung siku []. Jika tidak ditemukan, maka program akan menampilkan pesan bahwa data tidak ada dalam linked list.

Pada baris [266] sampai [312] terdapat fungsi hapusData() yang digunakan untuk menghapus semua node yang memiliki nilai tertentu sesuai dengan input pengguna. Fungsi ini akan menelusuri linked list dan menghapus setiap node yang memiliki nilai yang sama dengan data yang dimasukkan.

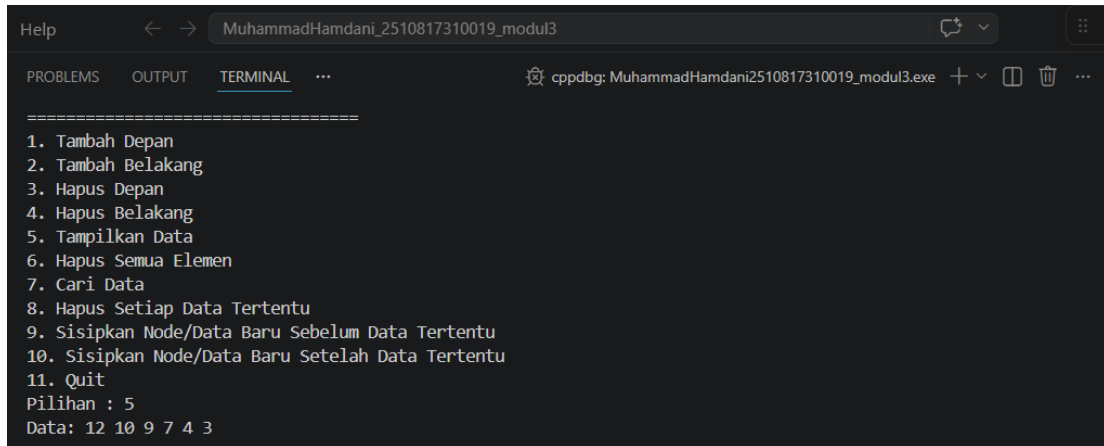
Pada baris [314] sampai [348] terdapat fungsi sisipkanSebelum() yang digunakan untuk menyisipkan node baru sebelum data tertentu. Program akan mencari node target terlebih dahulu. Jika ditemukan, maka node baru akan disisipkan sebelum node tersebut. Jika node target adalah head, maka node baru akan menjadi head yang baru.

Pada baris [350] sampai [382] terdapat fungsi sisipkanSetelah() yang digunakan untuk menyisipkan node baru setelah data tertentu. Jika node target ditemukan, maka node baru akan ditambahkan setelah node tersebut. Jika node target adalah tail, maka node baru akan menjadi tail yang baru.

SOAL 2

Lakukan tambah data depan 3, 4, 7, 9, 10, 12 dan kemudian lakukan tampilkan data lalu screenshoot hasilnya !

Output Program



```
Help  <  >  MuhammadHamdani_2510817310019_modul3  [Share] [Menu]
PROBLEMS  OUTPUT  TERMINAL  ...  cplusplus: MuhammadHamdani2510817310019_modul3.exe  [+][v][Icon][Icon]
=====
1. Tambah Depan
2. Tambah Belakang
3. Hapus Depan
4. Hapus Belakang
5. Tampilkan Data
6. Hapus Semua Elemen
7. Cari Data
8. Hapus Setiap Data Tertentu
9. Sisipkan Node/Data Baru Sebelum Data Tertentu
10. Sisipkan Node/Data Baru Setelah Data Tertentu
11. Quit
Pilihan : 5
Data: 12 10 9 7 4 3
```

Gambar 2. Tampilan Nilai Yang Diinput Dari Depan

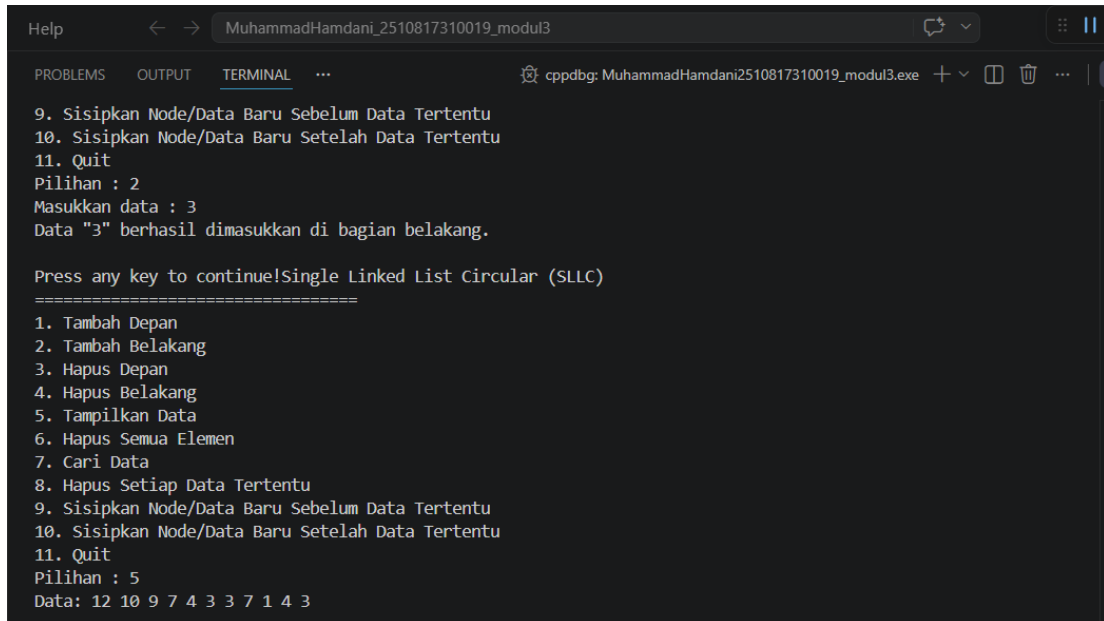
Pembahasan

Ketika data dimasukkan ke dalam Linked List melalui bagian depan, dengan urutan masuk mulai dari 3, 4, 7, 9, 10, dan 12. Angka 3 yang pertama kali dimasukkan akan muncul di sebelah kiri ketika di tampilkan. Data-data yang muncul setelah data pertama ditambahkan akan menjadi head dari Linked List dan data sebelumnya akan digeser ke samping kiri atau ke belakang.

SOAL 3

Lakukan tambah data belakang 3, 7, 1, 4, 3 dan kemudian lakukan tampilkan data lalu screenshoot hasilnya !

Output Program



```
Help    <  >    MuhammadHamdani_2510817310019_modul3
PROBLEMS  OUTPUT  TERMINAL  ...
cppdbg: MuhammadHamdani2510817310019_modul3.exe +  ▾  □  🗑  ...

9. Sisipkan Node/Data Baru Sebelum Data Tertentu
10. Sisipkan Node/Data Baru Setelah Data Tertentu
11. Quit
Pilihan : 2
Masukkan data : 3
Data "3" berhasil dimasukkan di bagian belakang.

Press any key to continue!Single Linked List Circular (SLLC)
=====
1. Tambah Depan
2. Tambah Belakang
3. Hapus Depan
4. Hapus Belakang
5. Tampilkan Data
6. Hapus Semua Elemen
7. Cari Data
8. Hapus Setiap Data Tertentu
9. Sisipkan Node/Data Baru Sebelum Data Tertentu
10. Sisipkan Node/Data Baru Setelah Data Tertentu
11. Quit
Pilihan : 5
Data: 12 10 9 7 4 3 3 7 1 4 3
```

Gambar 3. Tampilan Yang Diinput Dari Belakang

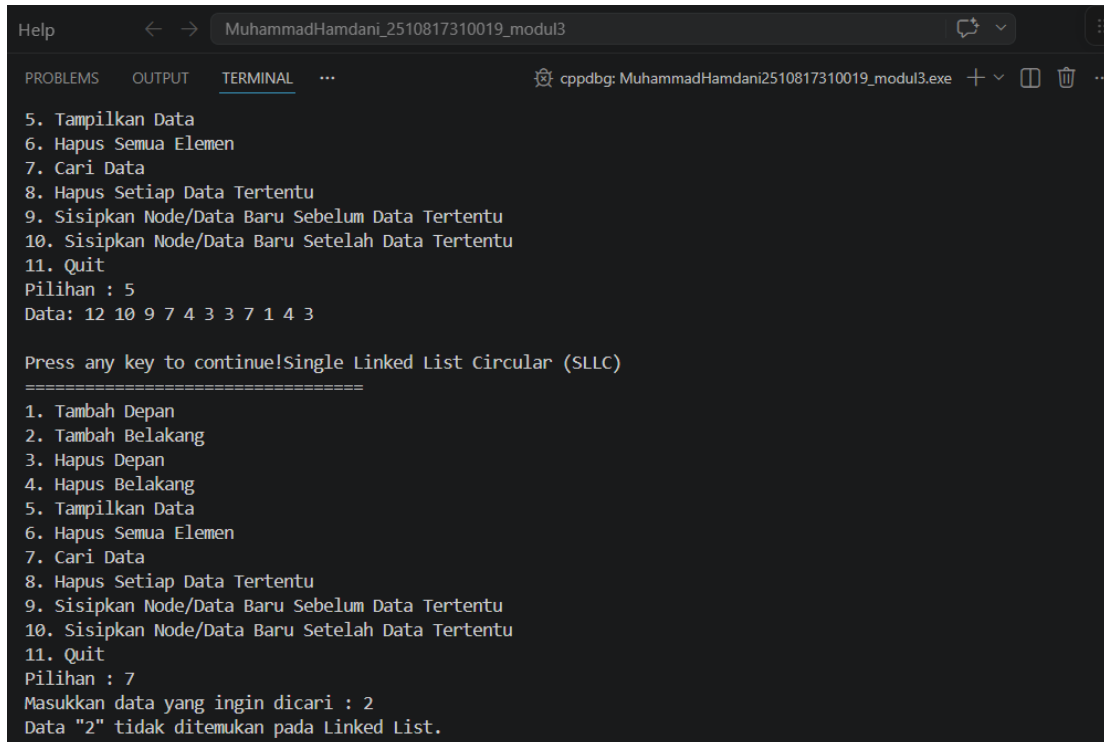
Pembahasan

Ketika data dimasukkan ke dalam Linked List melalui bagian belakang, dengan urutan masuk mulai dari 3, 7, 1, 4, dan 3. Angka 3 yang pertama kali dimasukkan akan muncul di sebelah kiri ketika di tampilkan. Data-data yang muncul setelah data pertama ditambahkan akan menjadi tail dari Linked List dan tampilkan dari Linked List akan sama seperti saat kita memasukkan data ke dalam Linked List.

SOAL 4

Apa yang terjadi jika mencari angka 2 pada Single Linked List Circular (SLLC) pada data yang telah ditambahkan/dimasukkan sebelumnya dan screenshoot hasilnya

Output Program



```
Help  ← → MuhammadHamdani_2510817310019_modul3
PROBLEMS OUTPUT TERMINAL ... cplusplus: MuhammadHamdani2510817310019_modul3.exe + - [ ] [X] ...

5. Tampilkan Data
6. Hapus Semua Elemen
7. Cari Data
8. Hapus Setiap Data Tertentu
9. Sisipkan Node/Data Baru Sebelum Data Tertentu
10. Sisipkan Node/Data Baru Setelah Data Tertentu
11. Quit
Pilihan : 5
Data: 12 10 9 7 4 3 3 7 1 4 3

Press any key to continue!Single Linked List Circular (SLLC)
=====
1. Tambah Depan
2. Tambah Belakang
3. Hapus Depan
4. Hapus Belakang
5. Tampilkan Data
6. Hapus Semua Elemen
7. Cari Data
8. Hapus Setiap Data Tertentu
9. Sisipkan Node/Data Baru Sebelum Data Tertentu
10. Sisipkan Node/Data Baru Setelah Data Tertentu
11. Quit
Pilihan : 7
Masukkan data yang ingin dicari : 2
Data "2" tidak ditemukan pada Linked List.
```

Gambar 4. Pencarian Nilai 2 pada Linked List

Pembahasan

Berdasarkan data yang telah dimasukkan pada soal 2 dan soal 3, ketika dilakukan sebuah pencarian untuk nilai 2. Sistem akan memberikan informasi bahwa nilai tersebut tidak terdapat atau tidak ditemukan pada Linked List.

SOAL 5

Coba cari angka 7 dan screenshoot hasilnya !

Output Program

```
=====
1. Tambah Depan
2. Tambah Belakang
3. Hapus Depan
4. Hapus Belakang
5. Tampilkan Data
6. Hapus Semua Elemen
7. Cari Data
8. Hapus Setiap Data Tertentu
9. Sisipkan Node/Data Baru Sebelum Data Tertentu
10. Sisipkan Node/Data Baru Setelah Data Tertentu
11. Quit
Pilihan : 7
Masukkan data yang ingin dicari : 7
Setiap data yang berada di dalam tanda kurung siku ([...]) merupakan data yang anda cari.
Linked List : 12 10 9 [7] 4 3 3 [7] 1 4 3
```

Gambar 5. Pencarian Nilai 7 pada Linked List

Pembahasan

Berdasarkan data yang telah dimasukkan pada soal 2 dan soal 3, ketika dilakukan sebuah pencarian untuk nilai 7. Sistem akan memberikan informasi bahwa nilai yang dicari akan ditandai dengan kurung siku [...] pada Linked List.

SOAL 6

Lakukan hapus belakang dan kemudian lakukan tampilkan data lalu screenshoot hasilnya !

Output Program

```

Help  ← → MuhammadHamdani_2510817310019_modul3
PROBLEMS OUTPUT TERMINAL ... cppdbg: MuhammadHamdani2510817310019_modul3.exe + ▾ 📄 🗑️ ...

9. Sisipkan Node/Data Baru Sebelum Data Tertentu
10. Sisipkan Node/Data Baru Setelah Data Tertentu
11. Quit
Pilihan : 7
Masukkan data yang ingin dicari : 7
Setiap data yang berada di dalam tanda kurung siku ([...]) merupakan data yang anda cari.
Linked List : 12 10 9 [7] 4 3 3 [7] 1 4 3

Press any key to continue!Single Linked List Circular (SLLC)
=====
1. Tambah Depan
2. Tambah Belakang
3. Hapus Depan
4. Hapus Belakang
5. Tampilkan Data
6. Hapus Semua Elemen
7. Cari Data
8. Hapus Setiap Data Tertentu
9. Sisipkan Node/Data Baru Sebelum Data Tertentu
10. Sisipkan Node/Data Baru Setelah Data Tertentu
11. Quit
Pilihan : 4
Data "3" yang berada di belakang telah berhasil dihapus.

Press any key to continue!Single Linked List Circular (SLLC)
=====
1. Tambah Depan
2. Tambah Belakang
3. Hapus Depan
4. Hapus Belakang
5. Tampilkan Data
6. Hapus Semua Elemen
7. Cari Data
8. Hapus Setiap Data Tertentu
9. Sisipkan Node/Data Baru Sebelum Data Tertentu
10. Sisipkan Node/Data Baru Setelah Data Tertentu
11. Quit
Pilihan : 5
Data: 12 10 9 7 4 3 3 7 1 4

```

Gambar 6. Tampilan Linked List Setelah Di Hapus

Pembahasan

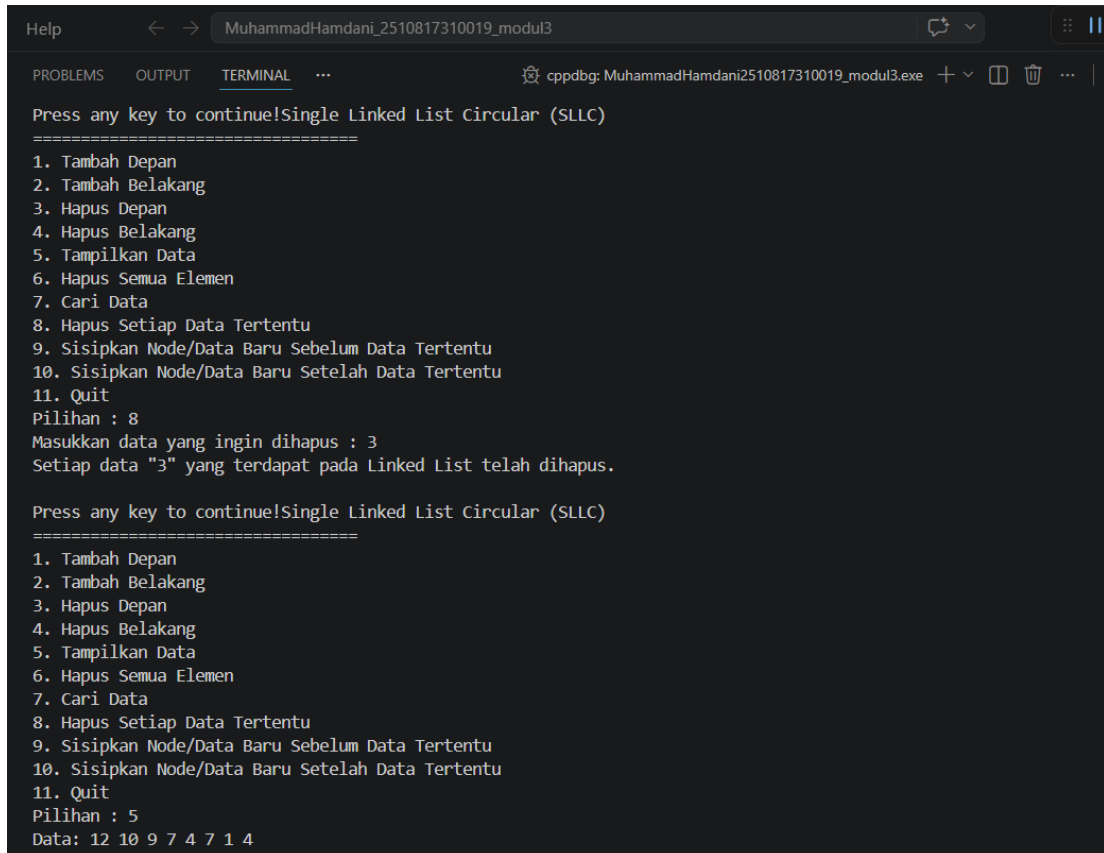
Berdasarkan data yang telah dimasukkan pada soal 2 dan soal 3, ketika dilakukan sebuah penghapusan data untuk bagian belakang atau tail. Nilai 3 yang terletak di bagian paling belakang pada Linked List (tail) akan dihapuskan dan ketika Linked List di tampilkan nilai yang ada disebelah kiri dari nilai yang dihapus akan menjadi

tail yang baru.

SOAL 7

Lakukan hapus setiap angka 3 dan kemudian lakukan tampilkan data lalu screenshoot hasilnya !

Output Program



```
Help    <  >    MuhammadHamdani_2510817310019_modul3
PROBLEMS  OUTPUT  TERMINAL  ...
cppdbg: MuhammadHamdani2510817310019_modul3.exe + v  [ ] [ ] ...

Press any key to continue!Single Linked List Circular (SLLC)
=====
1. Tambah Depan
2. Tambah Belakang
3. Hapus Depan
4. Hapus Belakang
5. Tampilkan Data
6. Hapus Semua Elemen
7. Cari Data
8. Hapus Setiap Data Tertentu
9. Sisipkan Node/Data Baru Sebelum Data Tertentu
10. Sisipkan Node/Data Baru Setelah Data Tertentu
11. Quit
Pilihan : 8
Masukkan data yang ingin dihapus : 3
Setiap data "3" yang terdapat pada Linked List telah dihapus.

Press any key to continue!Single Linked List Circular (SLLC)
=====
1. Tambah Depan
2. Tambah Belakang
3. Hapus Depan
4. Hapus Belakang
5. Tampilkan Data
6. Hapus Semua Elemen
7. Cari Data
8. Hapus Setiap Data Tertentu
9. Sisipkan Node/Data Baru Sebelum Data Tertentu
10. Sisipkan Node/Data Baru Setelah Data Tertentu
11. Quit
Pilihan : 5
Data: 12 10 9 7 4 7 1 4
```

Gambar 7. Tampilan Linked List Setelah Setiap Nilai 3 Dihapus

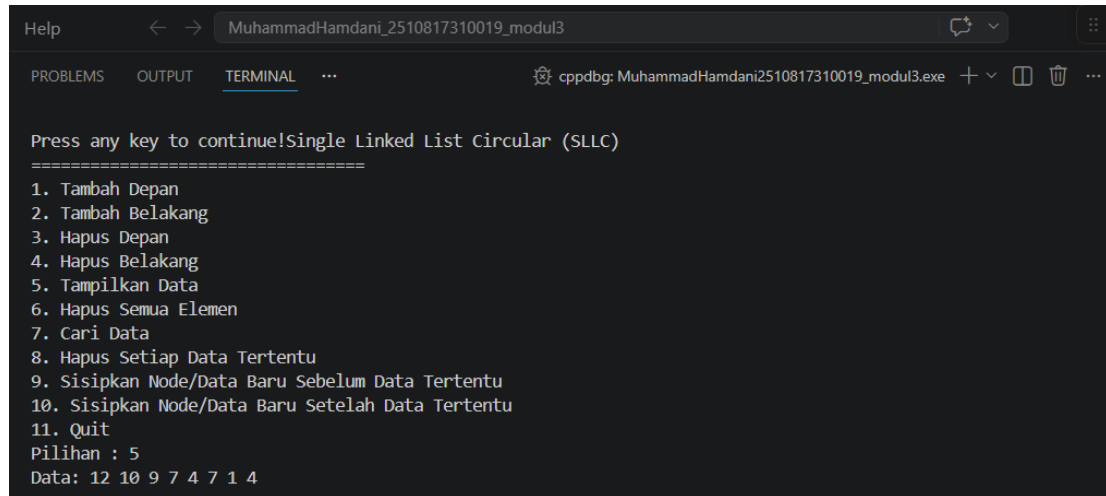
Pembahasan

Berdasarkan data yang telah dimasukkan pada soal 2 dan soal 3, ketika dilakukan sebuah penghapusan untuk setiap Nilai 3 yang ada pada Linked List. Nilai 3 yang terdapat pada Linked List akan dihapuskan semua dan ketika Linked List di tampilkan tidak akan terdapat nilai 3 di dalamnya.

SOAL 8

Tampilkan data lalu jelaskan yang mana head dan yang mana tail.

Output Program



```
Help    <  >    MuhammadHamdani_2510817310019_modul3
PROBLEMS  OUTPUT  TERMINAL  ...
cppdbg: MuhammadHamdani2510817310019_modul3.exe + v  [ ] [ ] ...

Press any key to continue!Single Linked List Circular (SLLC)
=====
1. Tambah Depan
2. Tambah Belakang
3. Hapus Depan
4. Hapus Belakang
5. Tampilkan Data
6. Hapus Semua Elemen
7. Cari Data
8. Hapus Setiap Data Tertentu
9. Sisipkan Node/Data Baru Sebelum Data Tertentu
10. Sisipkan Node/Data Baru Setelah Data Tertentu
11. Quit
Pilihan : 5
Data: 12 10 9 7 4 7 1 4
```

Gambar 8. Tampilan Isi dari Linked List

Pembahasan

Berdasarkan data yang telah dimasukkan pada soal 2 dan soal 3, kemudian dilakukannya penghapusan pada soal 6 dan soal 7. Data yang berada di paling depan atau di samping kiri akan menjadi head yaitu 12, dapat juga dibuktikan dengan melakukan sebuah operasi penambahan atau penghapusan data dari depan yang mana akan mempengaruhi data atau nilai dari 12 itu sendiri. Apabila ditambah data 1 pada Linked List dari depan data yang baru ditambahkan tersebut yang akan menjadi head. Begitu juga dengan penghapusan data atau nilai dari 12, maka data atau nilai yang ada di belakangnya yang akan menjadi head baru. Sedangkan data yang berada di paling belakang atau di samping kanan akan menjadi tail yaitu 4, dapat juga dibuktikan dengan melakukan sebuah operasi penambahan atau penghapusan data dari belakang yang mana akan mempengaruhi data atau nilai dari 4 itu sendiri. Apabila ditambah data 1 pada Linked List dari belakang data yang baru ditambahkan tersebut yang akan menjadi tail. Begitu juga dengan penghapusan data atau nilai dari 4, maka data atau nilai yang ada di depannya yang akan menjadi tail

baru.

SOAL 9

Jika baris ke 103 dan 104 dihapus maka apa yang akan terjadi pada saat memasukkan data, dan jelaskan mengapa?

Pembahasan

Apabila baris ke-103 dan 104 dihapuskan dari program yang mana pada baris itu terdapat *if(head == NULL) return 1;* dan *else return 0;* di dalamnya. Maka akan fungsi pengecekan untuk mengetahui apakah Linked List kosong atau berisi tidak bisa dilakukan di dalam program. Tanpa kedua baris kode tersebut, program masih bisa berjalan, namun berisiko mengalami kesalahan atau perilaku yang tidak terduga, seperti penggunaan garbage value atau pengaksesan memori yang tidak valid, yang dapat menyebabkan crash atau error lainnya.

SOAL 10

Jelaskan apa itu variabel head dan tail pada sllc!

Pembahasan

Pada Single Linked List Circular (SLLC), variabel head merupakan pointer yang menunjuk ke node pertama dalam Linked List, atau dengan kata lain, variabel head berfungsi sebagai tempat pertama data akan ditelusuri. Sedangkan variabel tail merupakan pointer yang menunjuk ke node terakhir dalam Linked List, atau tempat terakhir data akan ditelusuri.

Karena SLLC bersifat melingkar (*circular*), maka node terakhir (*tail*) akan menunjuk kembali ke node pertama (*head*), yang mana akan menjadikannya sebuah struktur yang terus berputar. Singkatnya, tail sebagai node terakhir akan menunjuk kembali ke head, sehingga memungkinkan traversal list secara terus-menerus tanpa ada akhir.

TAUTAN GITHUB

https://github.com/2510817310019-maker/Laprak-Algoritma-dan-Struktur-Data-muhammadhamdani-2510817310019/blob/main/MuhammadHamdani2510817310019_modul3.cpp